

```

class Machine {
    final static int
        CST = 0, VAR = 1, ADD = 2, SUB = 3,
        MUL = 4, POP = 5, SWAP = 6;
    static int seval(int[] code) {
        int[] stack = new int[1000];           // eval and env stack
        int sp = -1;                          // stack top pointer
        int pc = 0;                           // program counter
        int instr;                             // current instruction
        while (pc < code.length)
            switch (instr = code[pc++]) {
                case CST:
                    stack[sp+1] = code[pc++]; sp++; break;
                case VAR:
                    stack[sp+1] = stack[sp-code[pc++]]; sp++; break;
                case ADD:
                    stack[sp-1] = stack[sp-1] + stack[sp]; sp--; break;
                case SUB:
                    stack[sp-1] = stack[sp-1] - stack[sp]; sp--; break;
                case MUL:
                    stack[sp-1] = stack[sp-1] * stack[sp]; sp--; break;
                case POP:
                    sp--; break;
                case SWAP:
                    { int tmp = stack[sp];
                      stack[sp] = stack[sp-1];
                      stack[sp-1] = tmp;
                      break;
                    }
                default: ... error: unknown instruction ...
            }
        return stack[sp];
    }
}

```

Fig. 2.4 Stack machine in Java for expression evaluation (file `Machine.java`)

Exercise 2.1 Extend the expression language `expr` from `Intcomp1.fs` with multiple *sequential* let-bindings, such as this (in concrete syntax):

```
let x1 = 5+7    x2 = x1*2 in x1+x2 end
```

To evaluate this, the right-hand side expression `5+7` must be evaluated and bound to `x1`, and then `x1*2` must be evaluated and bound to `x2`, after which the let-body `x1+x2` is evaluated.

The new abstract syntax for `expr` might be

```

type expr =
  | CstI of int
  | Var of string
  | Let of (string * expr) list * expr      (* CHANGED *)
  | Prim of string * expr * expr

```

so that the `Let` constructor takes a list of bindings, where a binding is a pair of a variable name and an expression. The example above would be represented as:

```
Let ([("x1", ...); ("x2", ...)], Prim("+", Var "x1", Var "x2"))
```

Revise the `eval` interpreter from `Intcomp1.fs` to work for the `expr` language extended with multiple sequential `let`-bindings.

Exercise 2.2 Revise the function `freevars : expr -> string list` to work for the language as extended in Exercise 2.1. Note that the example expression in the beginning of Exercise 2.1 has no free variables, but `let x1 = x1+7 in x1+8 end` has the free variable `x1`, because the variable `x1` is bound only in the body (`x1+8`), not in the right-hand side (`x1+7`), of its own binding. There *are* programming languages where a variable can be used in the right-hand side of its own binding, but ours is not such a language.

Exercise 2.3 Revise the `expr-to-texpr` compiler `tcomp : expr -> texpr` from `Intcomp1.fs` to work for the extended `expr` language. There is no need to modify the `texpr` language or the `teval` interpreter to accommodate multiple sequential `let`-bindings.

Exercise 2.4 Write a bytecode assembler (in F#) that translates a list of bytecode instructions for the simple stack machine in `Intcomp1.fs` into a list of integers. The integers should be the corresponding bytecodes for the interpreter in `Machine.java`. Thus you should write a function `assemble : sinstr list -> int list`.

Use this function together with `scomp` from `Intcomp1.fs` to make a compiler from the original expressions language `expr` to a list of bytecodes `int list`.

You may test the output of your compiler by typing in the numbers as an `int` array in the `Machine.java` interpreter. (Or you may solve Exercise 2.5 below to avoid this manual work).

Exercise 2.5 Modify the compiler from Exercise 2.4 to write the lists of integers to a file. An F# list `inss` of integers may be output to the file called `fname` using this function (found in `Intcomp1.fs`):

```
let intsToFile (inss : int list) (fname : string) =
    let text = String.concat " " (List.map string inss)
    System.IO.File.WriteAllText(fname, text)
```

Then modify the stack machine interpreter in `Machine.java` to read the sequence of integers from a text file, and execute it as a stack machine program. The name of the text file may be given as a command-line parameter to the Java program. Reading numbers from the text file may be done using the `StringTokenizer` class or `StreamTokenizer` class; see e.g. *Java Precisely* [4, Example 194].

It is essential that the compiler (in F#) and the interpreter (in Java) agree on the intermediate language: what integer represents what instruction.